Requirements Gathering Document

Project: PayLens - Real-Time Merchant Intelligence, Risk & Settlement Platform

---

1. Stakeholder Analysis

- **Fintech & PSP Operations Teams:** Require fault-tolerant infrastructure, low-latency ingestion, and automated reconciliation exception reporting.
- **E-Commerce Merchants:** Require secure, tenant-isolated dashboards displaying gross sales, net pending payouts, approval rates, and risk flags.
- **Compliance & Risk Officers:** Require auditable risk rule logs, chargeback tracking, and immutable financial ledgers.

---

2. Functional Requirements (FR)

- **FR-01 (Hybrid Ingestion):** The platform must ingest high-velocity streaming payment events (via Event Hubs) alongside batch merchant master data (via ADF).
- **FR-02 (Medallion Processing):** The system must process data through Bronze (raw storage), Silver (validated Delta tables), and Gold (OLAP fact/dimension tables) layers.
- **FR-03 (Explainable Risk Evaluation):** The system must evaluate velocity spikes and value anomalies, generating `REVIEW REQUIRED` alerts without blocking live funds.
- **FR-04 (Dynamic Settlement Calculation):** The system must compute daily merchant net payouts using append-only ledger rules (`Captured Sales - Refunds - Lost Chargebacks - Platform Fees`).
- **FR-05 (3-Way Reconciliation):** The system must automatically match internal records against processor transaction reports and bank payout statement line items.

---

3. Non-Functional Requirements (NFR)

- **NFR-01 (Idempotency & Deduplication):** The pipeline must handle duplicate event deliveries and late-arriving results gracefully without double-counting transactions.
- **NFR-02 (Data Immutability):** Financial settlement records and audit logs must follow append-only rules where historical data is never overwritten or deleted.
- **NFR-03 (Tenant Isolation):** Merchant analytics dashboards must strictly isolate multi-tenant data access.

- **NFR-04 (Version Control):** All codebases, event schemas, and data pipelines must be maintained on GitHub with clear commit documentation.

---

4. User Stories & Use Cases

User Story 1 (Data Engineer)

As a Data Engineer, I want to ingest streaming authorization and capture events into a Delta Lake Bronze layer so that I can process late-arriving and duplicate events safely using PySpark MERGE operations.

User Story 2 (Finance Manager)

As a Finance Manager, I want an automated 3-way reconciliation output so that I can instantly identify amount mismatches or missing bank payouts across processor logs and internal ledgers.